

Project Document: NYC Taxi Trip Duration Prediction

Index

1. Project Objective
2. Data Loading and Initial Exploration
3. Data Preprocessing and Feature Engineering
4. Model Training and Comparison
5. Data Drift Simulation and Monitoring
6. Improving Model Performance: Hyperparameter Tuning
7. Model Packaging and Serving
8. MLflow UI Access
9. Resources
10. Artifacts

1. Project Objective

The primary objective of this mini-project is to develop a machine learning model capable of predicting NYC taxi trip durations. Beyond basic model development, a significant focus is placed on demonstrating key MLOps practices, including MLflow for experiment tracking, model versioning, data drift simulation, and model serving through a self-contained Docker image. The goal is to build a robust and maintainable ML pipeline.

2. Data Loading and Initial Exploration

Process:

- The raw taxi trip data (NY_Trip_Raw_Data.csv) was loaded into a pandas DataFrame.
- Initial exploration involved using `df.info()` to inspect data types and non-null counts, `df.isnull().sum()` to identify missing values, and `df.describe()` to get descriptive statistics of numerical columns.

Observations:

- The dataset contains approximately 627,097 entries and 12 columns, including trip ID, vendor ID, pickup/dropoff datetimes, passenger count, weather conditions, GPS coordinates, store-and-forward flag, and trip duration.
- A small number of missing values (1 in most columns, 4 for GPS coordinates due to a single row) were identified.
- `pickup_datetime` and `dropoff_datetime` were initially loaded as objects (strings) and required conversion.

- `trip_duration` is the target variable, ranging from 1 second to over 1.9 million seconds, suggesting potential outliers.

3. Data Preprocessing and Feature Engineering

Process:

- **Schema Validation:**
 - `pickup_datetime` and `dropoff_datetime` columns were converted to datetime objects, with errors='coerce' to handle unparseable entries (none found).
 - Rows where `dropoff_datetime` was before `pickup_datetime` were removed (none found).
 - Rows with missing GPS pings (4 rows) were identified.
 - Rows with any invalid timestamps were dropped.
- **Feature Engineering:**
 - **`pickup_hour`:** Extracted the hour from `pickup_datetime`.
 - **`pickup_dayofweek`:** Extracted the day of the week (Monday=0, Sunday=6) from `pickup_datetime`.
 - **`is_weekend`:** A binary feature indicating if the `pickup_dayofweek` is Saturday or Sunday.
 - **`distance_km`:** Calculated the Haversine distance between pickup and dropoff coordinates.
 - **`weather`:** Converted to a categorical data type.
- **Dataset Versioning:** The processed data was saved to a new CSV file (`NY_Trip_Processed_Data_v1.csv`) for version control and reproducibility.

4. Model Training and Comparison

Process:

- **Data Preparation:** The `id`, original datetime columns, and `trip_duration` (target) were dropped from the feature set X. Categorical features (`weather`, `store_and_fwd_flag`, `vendor_id`) were one-hot encoded.
- The data was split into training (80%) and testing (20%) sets using `train_test_split`.

- **MLflow Integration:** MLflow was used to track experiments for both models, logging parameters, metrics (MSE, R-squared), and model artifacts.
- **Model 1: Linear Regression**
 - Trained a LinearRegression model on the cleaned training data.
 - Evaluated its performance on the cleaned test data, calculating MSE and R-squared.
 - Logged the model and its metrics to MLflow.
- **Model 2: Gradient Boosting Regressor**
 - Trained a GradientBoostingRegressor with default hyperparameters (`n_estimators=100`, `learning_rate=0.1`) on the cleaned training data.
 - Evaluated its performance on the test data, calculating MSE and R-squared.
 - Logged the model and its metrics to MLflow.

Results:

- **Linear Regression:** MSE = 9,516,939.78, R-squared = 0.03
- **Gradient Boosting Regressor:** MSE = 9,505,976.93, R-squared = 0.03

Conclusion: The Gradient Boosting Regressor showed a marginally better R-squared score, indicating slightly superior performance in explaining the variance in trip duration, albeit both models performed poorly with the given feature set.

5. Data Drift Simulation and Monitoring

Process:

- **Data Drift Simulation:** A 'rush hour surge' scenario was simulated on a copy of the original DataFrame (`df_drift`). This involved:
 - Increasing passenger counts during defined rush hour periods (7-9 AM, 4-6 PM) for a percentage of trips.
 - Slightly increasing `distance_km` for rush hour trips (simulating detours/traffic).
 - Increasing `trip_duration` for rush hour trips (simulating congestion).
- The `df_drift` was preprocessed and one-hot encoded to match the model's expected input features.

- **Performance Monitoring:** The trained Linear Regression model (`lr_model`) was used to make predictions on the `X_drift` data.
- MSE and R-squared were calculated for the model's performance on the drifted data.
- **Retraining Trigger:** A performance degradation threshold was defined (10% drop in R-squared from original performance).
- If the R-squared on drifted data fell below this threshold, a retraining process was simulated (retraining the `LinearRegression` model on the original `X_train_cleaned` and `y_train_cleaned`).
- The retrained model's performance was evaluated and logged to MLflow.

Results:

- R-squared on Drifted Data (Linear Regression): 0.0170
- Original R-squared (Linear Regression): 0.0279
- The R-squared value on drifted data fell below the defined threshold (0.0251), triggering a simulated retraining. The retrained model achieved an R-squared of 0.03, demonstrating recovery to original performance.

6. Improving Model Performance: Hyperparameter Tuning

Process:

- Recognizing the low R-squared scores, `GridSearchCV` was initiated for hyperparameter tuning of the `GradientBoostingRegressor`.
- A `param_grid` was defined with variations for `n_estimators`, `learning_rate`, and `max_depth`.
- `GridSearchCV` was configured with 3-fold cross-validation and `r2` as the scoring metric.
- **(Note: The actual `grid_search.fit` and subsequent steps were commented out in the notebook, indicating this was a demonstration of the *approach* rather than a fully executed tuning due to computational constraints for a mini-project.)**
- The intention was to log the best parameters and the performance of the tuned model to MLflow.

7. Model Packaging and Serving

Process:

- **Model Loading:** The best-performing model (Linear Regression based on the ADR's decision due to being the focus of subsequent steps, despite GBR's marginal edge) was loaded from MLflow using its run_id and model_uri.
- **Sample Prediction:** The loaded model was used to make predictions on a sample of test data to demonstrate its functionality.
- **REST API Serving (MLflow mlflow models serve):** Instructions were provided to serve the model locally as a REST API using the mlflow models serve command, requiring the MLflow run ID and specifying a port.
- **Dockerfile Generation (MLflow with Tracking Server):** A requirements.txt and Dockerfile were generated. This Dockerfile was designed to build an image that would fetch the model from an MLflow tracking server at runtime.
- **Dockerfile Generation (Self-Contained):** An alternative, self-contained Dockerfile was generated. This involved:
 - Downloading the model artifacts locally.
 - Copying these artifacts into the Docker image.
 - Modifying the Dockerfile to serve the model directly from the embedded artifacts, removing the dependency on an external MLflow tracking server at runtime.
- Detailed instructions for building and running both types of Docker images were provided.
- A sample JSON payload for testing the model endpoint was also provided.\

8. MLflow UI Access

Process:

- MLflow UI was started in the background on port 5000.
- The MLFLOW_TRACKING_URI was printed, which could be used to connect local MLflow clients or Docker containers to this remote tracking server.

9. Resources

This project utilized several open-source libraries and platforms:

- **pandas:** For data manipulation and analysis.

- **numpy:** For numerical operations.
- **scikit-learn:** For machine learning model implementations (Linear Regression, Gradient Boosting Regressor) and utility functions (train_test_split, GridSearchCV).
- **matplotlib & seaborn:** For data visualization.
- **MLflow:** For experiment tracking, model logging, model registry, and model serving utilities.
- **Docker:** For containerization of the ML model for deployment.

10. Artifacts:

Git Hub location: <https://github.com/2025paml563-coder/mlminiproject>

Versioned data: <https://github.com/2025paml563-coder/mlminiproject/blob/main/data.zip>

Feature engineered data: https://github.com/2025paml563-coder/mlminiproject/blob/main/data_feature_engineered.zip